

二分图问题

什么是二分图:

- 若图 $G = (V, E)$ 的顶点集 V 可分为两个非空子集 U 和 W ($U \cap W = \emptyset$, $U \cup W = V$) , 使得每条边 $e \in E$ 的两个端点分别属于 U 和 W , 则 G 为二分图, 记为 $G = (U, W, E)$ 。
- 二分图中不存在奇数长度的环 (如三角形) 。若图中存在奇数环, 则必不是二分图, 这是二分图的重要性质。

举例:

学生选课关系图

图的构造

- 顶点集合:**
 - U = 所有学生 (如学生 A、B、C) ,
 - W = 所有课程 (如课程 X、Y、Z) 。
- 边的定义:** 若学生选了某门课, 则学生与课程之间连一条边。

二分图验证:

- 顶点二分性:** 学生和课程是两类不同的对象, 集合 U 和 W 无交集。
- 边的跨组性:** 边只存在于 “学生→课程” 之间, 不存在 “学生→学生” 或 “课程→课程” 的边 (选课关系不涉及学生之间或课程之间的直接连接) 。
- 无奇数环:** 假设学生 A 选了课程 X, 课程 X 被学生 B 选, 学生 B 选了课程 Y, 课程 Y 被学生 A 选, 形成环 “学生 A→课程 X→学生 B→课程 Y→学生 A”, 环的长度为 4 (偶数) , 不存在奇数环 (如 3 个顶点的环: 学生 A→课程 X→学生 B→学生 A, 这种情况不可能出现, 因为学生之间没有边) 。

判断方法:

1. 染色法 (常用判定算法)

- 从任意未染色顶点出发, 将其染为颜色 A, 然后将相邻顶点染为颜色 B, 再将相邻顶点的相邻顶点染为颜色 A, 以此类推。

- 若染色过程中发现相邻顶点颜色相同，则图中存在奇数环，不是二分图；否则是二分图。
- 时间复杂度： $(O(V + E))$ ，适用于所有图。

2. **数学性质判定** 图是二分图当且仅当它没有奇数长度的环（如 3 - 环、5 - 环等）。

染色法实现:

```
vector<int> g[N];
int color[N];
bool bfs()
{
    for(int i=1;i<=n;i++)
        //图不是一张连通图的时候需要查看每个节点
        {
            if(color[i]) continue;
            queue<int> q;
            color[i] = 1;
            q.push(i);
            while(!q.empty())
            {
                int t = q.front();
                q.pop();

                for(auto v : g[t])
                {
                    if(!color[v])
                    {
                        color[v] = color[t]%2+1;
                        q.push(v);
                    }
                    else if(color[v] == color[t]) return false;
                }
            }
        }
    return true;
}
```

例题:

P1330 封锁阳光大学

判定二分图 + 求最小点覆盖

本题要求用最少的蟹蟹封锁所有道路且不冲突，可转化为二分图最小顶点覆盖问题。首先需判断图是否为二分图，若存在奇数环则输出 "Impossible"。若为二分图，根据二分图性质，最小顶点覆盖

盖数为两部分节点数的较小值。具体步骤：用 BFS 染色法判定二分图，染色时记录两部分节点数，对每个连通块取较小部分节点数累加，即为最少河蟹数。

STD:

```

#include <bits/stdc++.h>
#define endl "\n"
#define int long long
using namespace std;
const int N = 1e5+10;

int n,m;
vector<int> g[N];
int color[N];
int tot1,tot2;
bool bfs(int x)
{
    queue<int> q;
    color[x] = 1;
    tot1 = 1;
    tot2 = 0;
    q.push(x);
    while(!q.empty())
    {
        int t = q.front();
        q.pop();

        for(auto v : g[t])
        {
            if(!color[v])
            {
                color[v] = color[t]%2+1;
                color[v] == 1 ? tot1++ : tot2++;
                q.push(v);
            }
            else if(color[v] == color[t]) return false;
        }
    }
    return true;
}

int ans;
void solve()
{
    cin >> n >> m;
    for(int i=1;i<=m;i++)
    {
        int u,v;cin >> u >> v;
        g[u].push_back(v);
        g[v].push_back(u);
    }
    for(int i=1;i<=n;i++)
    {
        if(color[i]) continue;
        if(!bfs(i))

```

```

        {
            cout << "Impossible" << endl;
            return;
        }
        else ans += min(tot1,tot2);
    }
    cout << ans << endl;
}

signed main()
{
    ios::sync_with_stdio(0);
    cin.tie(0), cout.tie(0);
    int _ = 1;
    //cin >> _;
    while(_--)
    {
        solve();
    }
    return 0;
}

```